Ecole Publique d'Ingénieurs en 3 ans

Rapport

OUTILS ET MÉTHODES DE QUANTIFICATION DES RÉSEAUX DE NEURONES CONVOLUTIFS (CNN)

3A 2025/2026

DEFORGE Arthur,
FADLOU Jeremy

GPSE / SATE
2025





TABLE DES MATIÈRES

1. Quantization **9**
1.1. Bref Rappel 9
1.1.1. Qu'est-ce que la quantisation ? 9
1.1.2. Les deux grandes approches : PTQ vs QAT 9
1.1.3. Quantisation dynamique vs statique 10
1.2. Disclaimer 10
2. AIMET[1,2,3,4,5,6,7,8,9,10,11] **11**
2.1. Adaptive Rounding (AdaRound) 11
2.1.1. Principe 11
2.1.2. Équations 11
2.1.3. Entrées / Sorties 12
2.1.4. Algorithme simplifié 12
2.1.5. Avantages 12
2.1.6. Défauts 12
2.1.7. Applications 13
2.2. Sequential MSE (SeqMSE) 14
2.2.1. Principe 14
2.2.2. Équations 14
2.2.3. Entrées / Sorties 14
2.2.4. Algorithme simplifié 14
2.2.5. Avantages 15
2.2.6. Défauts 15
2.2.7. Applications 15
2.3. Batch Norm Folding (BNF) 16
2.3.1. Principe 16
2.3.2. Équations 16
2.3.3. Entrées / Sorties 16
2.3.4. Algorithme simplifié 17
2.3.5. Avantages 17

2.3.6. Défauts....17
2.3.7. Applications....17
2.4. Cross-Layer Equalization (CLE)....18
2.4.1. Principe....18
2.4.2. Équations....18
2.4.3. Entrées / Sorties....18
2.4.4. Algorithme simplifié....18
2.4.5. Avantages....18
2.4.6. Défauts....19
2.4.7. Applications....19
2.5. AdaScale....20
2.5.1. Principe....20
2.5.2. Équations....20
2.5.3. Entrées / Sorties....20
2.5.4. Algorithme simplifié....20
2.5.5. Avantages....21
2.5.6. Défauts....21
2.5.7. Applications....21
2.6. OmniQuant....22
2.6.1. Principe....22
2.6.2. Équations....22
2.6.3. Entrées / Sorties....22
2.6.4. Algorithme simplifié....22
2.6.5. Avantages....22
2.6.6. Défauts....23
2.6.7. Applications....23
2.7. Batch Norm Re-estimation....24
2.7.1. Principe....24
2.7.2. Équations....24
2.7.3. Entrées / Sorties....24
2.7.4. Algorithme simplifié....24

2.7.5. Avantages....24
2.7.6. Défauts....24
2.7.7. Applications....25
2.8. SpinQuant....26
2.8.1. Principe....26
2.8.2. Équations....26
2.8.3. Entrées / Sorties....26
2.8.4. Algorithme simplifié....26
2.8.5. Avantages....26
2.9. Défauts....27
2.9.1. Applications....27
2.10. Automatic Quantization (AutoQuant)....28
2.10.1. Principe....28
2.10.2. Équations....28
2.10.3. Entrées / Sorties....28
2.10.4. Algorithme simplifié....28
2.10.5. Avantages....28
2.10.6. Défauts....29
2.10.7. Applications....29
3. MQBench[12,13,14]....30
3.1. Adaptive Rounding (AdaRound)....30
3.1.1. Principe....30
3.1.2. Équations....30
3.1.3. Entrées / Sorties....30
3.1.4. Algorithme simplifié....30
3.1.5. Avantages....31
3.1.6. Défauts....31
3.1.7. Applications....31
3.2. Block Reconstruction Quantization (BRECQ)....32
3.2.1. Principe....32
3.2.2. Équations....32

3.2.3. Entrées / Sorties....32
3.2.4. Algorithme simplifié....32
3.2.5. Avantages....33
3.2.6. Défauts....33
3.2.7. Applications....33
3.3. Quantization with Dropout (QDrop)....34
3.3.1. Principe....34
3.3.2. Équations....34
3.3.3. Entrées / Sorties....34
3.3.4. Algorithme simplifié....34
3.3.5. Avantages....35
3.3.6. Défauts....35
3.3.7. Applications....35
4. ONNX Runtime[15,16,17,18]....36
4.1. MinMax Quantization....36
4.1.1. Principe....36
4.1.2. Équations....36
4.1.3. Entrées / Sorties....36
4.1.4. Algorithme simplifié....36
4.1.5. Avantages....37
4.1.6. Défauts....37
4.1.7. Applications|....37
4.2. Entropy (KL Divergence) Quantization....38
4.2.1. Principe....38
4.2.2. Équations....38
4.2.3. Entrées / Sorties....38
4.2.4. Algorithme simplifié....38
4.2.5. Avantages....39
4.2.6. Défauts....39
4.2.7. Applications....39
5. Microsoft NNI[33]....40
|

5.1. Naive Quantizer....40
5.1.1. Principe....40
5.1.2. Équations....40
5.1.3. Entrées / Sorties....41
5.1.4. Algorithme simplifié....41
5.1.5. Avantages....41
5.1.6. Défauts....41
5.1.7. Applications....41
5.2. DoReFa Quantizer....42
5.2.1. Principe....42
5.2.2. Équations....42
5.2.3. Entrées / Sorties....42
5.2.4. Algorithme simplifié....42
5.2.5. Avantages....42
5.2.6. Défauts....43
5.2.7. Applications....43
5.3. BNN Quantizer....44
5.3.1. Principe....44
5.3.2. Équations....44
5.3.3. Entrées / Sorties....44
5.3.4. Algorithme simplifié....44
5.3.5. Avantages....44
5.3.6. Défauts....45
5.3.7. Applications....45
6. Neural Magic SparseML [34,35]....46
6.1. Introduction....46
6.2. Origine, objectifs et positionnement....46
6.2.1. Principales fonctionnalités....46
6.3. Conclusion....46
7. Nebullvm[36]....47
7.1. Introduction....47

7.2. Origine, objectifs et positionnement....47
7.2.1. Principales fonctionnalités....47
7.3. Conclusion....47
8. AMD Quark[37,38]....48
8.1. Introduction....48
8.2. Origine, objectifs et positionnement....48
8.2.1. Principales fonctionnalités....48
8.3. Conclusion....48
9. PyTorch[20,21,22]....49
9.1. Origine et contexte....49
9.2. Pourquoi utiliser PyTorch ?....49
9.3. Domaines d'application....49
9.4. Architecture, composants et réglages dans PyTorch....49
9.4.1. Principaux composants....49
9.4.2. Mise en œuvre d'un CNN....50
9.4.3. Réglages et optimisations spécifiques....50
9.4.4. Compatibilité d'architectures CNN....51
9.5. Synthèse....51
9.6. Conclusion....51
10. INC[23,24,25,26]....52
10.1. Présentation....52
10.2. Qu'est-ce que Intel Neural Compressor....52
10.3. Fonctionnalités principales....52
10.4. Techniques d'optimisations Post-Training :....52
10.5. Intégration avec PyTorch (points pratiques)....54
10.6. Conclusion....54
11. OpenVINO[27,28,29]....55
11.1. Présentation de OpenVINO Toolkit....55
11.1.1. Origine et contexte....55
11.1.2. Pourquoi utiliser OpenVINO ?....55
11.1.3. Domaines d'application....56

11.2. Architecture, composants et réglages dans OpenVINO....56
11.2.1. Principaux composants....56
11.2.2. Mise en œuvre d'un CNN....57
11.2.3. Réglages et optimisations spécifiques....57
11.2.4. Compatibilité d'architectures CNN....57
11.3. Synthèse....58
11.4. Conclusion....58
12. Brevitas[30,31,32]....59
12.1. Introduction....59
12.2. Origine, objectifs et positionnement....59
12.3. Principales fonctionnalités (ce que Brevitas apporte)....59
12.4. Architectures et cas d'usage ciblés....60
12.5. Réglages importants à maîtriser (pratiques)....60
12.6. Limites et points de vigilance....61
12.7. Conclusion....61
13. Projet « Awesome Model Quantization »[19]....62
13.1. Introduction....62
13.2. Objectif du dépôt GitHub....62
13.3. Structure du github....63
13.3.1. README :....63
13.3.2. Table des matières :....63
13.3.3. Méthodes de quantification :....63
13.3.4. Frameworks et Outils :....63
13.3.5. Références académiques :....63
13.3.6. Projets associés :....64
13.4. Revue des méthodes PTQ....64
13.4.1. Exemples de méthodes PTQ:....64
13.4.2. Benchmarks :....64
13.4.3. Hardware-aware Quantization :....65
13.4.4. Conclusion....65

1. Quantization

1.1. Bref Rappel

1.1.1. Qu'est-ce que la quantisation ?

La quantisation consiste à réduire la précision numérique des poids et/ou activations d'un modèle (par ex. de float32 → int8), pour :

- accélérer l'inférence (surtout sur CPU / edge devices),
- réduire la taille du modèle,
- avec une perte de précision minimale si bien faite.

1.1.2. Les deux grandes approches : PTQ vs QAT

Type	Nom complet	Moment d'application	Avantages	Inconvénients
PTQ	Post-Training Quantization	Après l'entraînement	Simple, rapide, pas besoin de réentraîner	Légère baisse de précision, surtout sur modèles sensibles
QAT	Quantization-Aware Training	Pendant l'entraînement	Meilleure précision finale	Nécessite du réentraînement et plus de calculs

Exemple :

- PTQ : tu prends un modèle pré-entraîné, tu le quantifies directement.
- QAT : tu simules la quantisation pendant l'entraînement, donc le modèle apprend à être robuste à la perte de précision.

1.1.3. Quantisation dynamique vs statique

Type	Description
Dynamique	Seuls les poids sont quantifiés à l'avance ; les activations sont quantifiées au moment de l'inférence (dynamiquement)
Statique	Poids et activations sont quantifiés avant l'inférence à partir d'un jeu de calibration

- Statique → besoin d'un petit dataset de calibration pour estimer les min/max des activations.
- Dynamique → plus simple, mais moins optimisé.

1.2. Disclaimer

Ce rapport se concentre exclusivement sur les méthodes de quantification post-entraînement (**PTQ**), en privilégiant les approches statiques. Il s'articule comme suit :

D'abord, sont présentés les **frameworks** dont les méthodes sont **clairement différenciées** : AIMET, Brevitas et MQBench.

Ensuite, sont abordés les frameworks dont la méthode n'est **pas explicitement détaillée** dans la documentation, ceux qui **s'appuient sur des frameworks déjà analysés**, ou ceux qui proposent principalement des **méthodes paramétrables** par l'utilisateur.

2. AIMET[1,2,3,4,5,6,7,8,9,10,11]

AIMET (AI Model Efficiency Toolkit) est une boîte à outils open-source développée par Qualcomm Innovation Center.

Son objectif principal est d'optimiser les modèles d'intelligence artificielle pour les rendre plus performants et moins gourmands en ressources, notamment pour un déploiement sur des appareils de périphérie (edge devices) comme les smartphones.

Pour ce faire, AIMET utilise des techniques avancées de quantification et de compression qui réduisent la taille du modèle et accélèrent son exécution, tout en s'efforçant de préserver sa précision d'origine. Il est compatible avec les frameworks PyTorch et ONNX.

2.1. Adaptive Rounding (AdaRound)

2.1.1. Principe

AdaRound vise à améliorer la quantification des poids en ajustant la direction d'arrondi de chaque poids afin de minimiser la perte de précision due à la quantification. Contrairement à un arrondi classique (au plus proche), AdaRound apprend s'il vaut mieux arrondir vers le haut ou vers le bas, en s'appuyant sur des échantillons d'entrée (non étiquetés).

2.1.2. Équations

Pour chaque poids flottant w, la quantification classique donne : $\hat{w} = \lfloor \frac{w}{s} + 0.5 \rfloor \times s$

où :

• s = facteur d'échelle,

• $\lfloor x \rfloor$ = arrondi au plus proche entier.

AdaRound introduit un biais d'arrondi appris : $\hat{w} = \lfloor \frac{w}{s} + sigma(\alpha) \rfloor \times s$

avec :

• $sigma(\alpha)$ = fonction sigmoïde,

• α est optimisé par descente de gradient pour minimiser :$L=\|f(x;W)-f(x;\hat{W})\|^2$

où $f(x;W)$ est la sortie du modèle en précision flottante.

2.1.3. Entrées / Sorties

Élément Type

- Entrée tenseur de poids float32
- Sortie tenseur quantifié int8 ou int4
- Échelle float
- Décalage z entier (souvent nul pour poids symétriques)

2.1.4. Algorithme simplifié

1. Charger le modèle FP32 et les données non étiquetées.
2. Pour chaque couche :
    1. Calculer les valeurs de s.
    2. Initialiser α(aléatoire ou nul).
    3. Itérer :
        1. Quantifier les poids via hat $w=\lfloor\frac{w}{s}+sigma(\alpha)\rfloor\times s$.
        2. Calculer la perte L.
        3. Mettre à jour α par descente de gradient.
        4. Geler les poids quantifiés optimisés.

2.1.5. Avantages

Améliore la précision sans entraînement complet.

Exploite les données réelles pour adapter la quantification.

Compatibilité avec ONNX et PyTorch.

2.1.6. Défauts

Nécessite un échantillon de calibration représentatif.

Plus coûteux que la simple quantification statique (≈ 10–15 k itérations).

2.1.7. Applications

Réduction de perte sur modèles sensibles (MobileNet, BERT).

Bon compromis entre vitesse et précision post-training.

2.2. Sequential MSE (SeqMSE)

2.2.1. Principe

SeqMSE cherche les meilleurs encodages de quantification pour chaque couche séquentiellement, en minimisant la Mean Squared Error (MSE) entre la sortie FP32 et la sortie quantifiée.

2.2.2. Équations

Pour chaque couche i : $L_i = \| y_i - \hat{y}_i \|^2$

où :

• y_i = sortie flottante de la couche i,

• $\hat{y}_i$ = sortie quantifiée selon les paramètres (s, z).

La conversion float → int est donnée par : $\hat{x} = s \cdot (\lfloor \frac{x}{s} + z \rfloor - z)$

2.2.3. Entrées / Sorties

Élément Type

- Entrée tenseur de poids float32 et activations
- Sortie tenseur quantifié int8 ou int4
- Échelle float
- Décalage z entier

2.2.4. Algorithme simplifié

1. Pour chaque couche i :
2. Fixer les couches précédentes quantifiées.
3. Générer N candidats (s, z).
4. Quantifier la couche avec chaque candidat.
5. Calculer L_i.
6. Choisir (s, z) minimisant L_i.

2.2.5. Avantages

- Ne requiert pas d’entraînement.
- Optimisation par recherche séquentielle locale.
- Robuste même avec peu de données.

2.2.6. Défauts

- Optimise couche par couche, non globalement optimal.
- Temps de calcul proportionnel au nombre de candidats.

2.2.7. Applications

- Convient aux modèles CNN ou MLP statiques.
- Recommandé lorsque peu de données sont disponibles.

2.3. Batch Norm Folding (BNF)

2.3.1. Principe

BNF fusionne les couches de normalisation par lots (BatchNorm) avec les couches de convolution ou linéaires adjacentes. Cela élimine les calculs redondants et améliore la précision de la quantification en réduisant les erreurs de requantification.

Batch (Lot)

- Définition simple : Sous-ensemble des données traité ensemble pendant l'entraînement.
- Définition mathématique :
  - Soit N la taille du dataset, B la taille du batch :
  - $X_b = x_i, x_{i+1}, \ldots, x_{i+B-1}$ *où i est l' indice de départ* .

2.3.2. Équations

Pour une convolution suivie d'un BatchNorm :

$$y = BN(Conv(x, W, b)) = \left(\frac{\gamma \cdot Conv(x, W, b) - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta\right)$$

En repliant :

$$\hat{W} = \frac{W \cdot \gamma}{\sqrt{\sigma^2 + \epsilon}} \; et \; \hat{b} = \frac{(b - \mu) \cdot \gamma}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

2.3.3. Entrées / Sorties

Élément Type

- Entrée Conv/Linear weight float32, BatchNorm (γ, e, μ, σ)
- Sortie tenseur de poids replié float32
- Échelle non applicable
- Décalage z non applicable

Tenseur de poids replié

- Définition simple : Poids fusionnés d'une convolution et du BatchNorm qui suit.

- Définition mathématique :
  - $\hat{W} = \frac{W \cdot \gamma}{\sqrt{\sigma^2 + \varepsilon}}$
  - $\hat{b} = \frac{(b - \mu) \cdot \gamma}{\sqrt{\sigma^2 + \varepsilon}} + \beta$

2.3.4. Algorithme simplifié

1. Pour chaque paire (Conv/Linear, BatchNorm) :
2. Parcourir le graphe du modèle.
3. Calculer $\hat{W}$ et $\hat{b}$.
4. Remplacer les poids de la convolution par $\hat{W}$ et $\hat{b}$.
5. Supprimer le nœud BatchNorm.

2.3.5. Avantages

- Réduction des opérations et de la mémoire.
- Améliore la précision de la quantification.
- Compatible avec les runtime d'inférence.

2.3.6. Défauts

- Modifie la structure du modèle.
- Non applicable si BatchNorm n'est pas adjacent à Conv/Linear.

2.3.7. Applications

- Modèles avec couches BatchNorm (ResNet, MobileNet).
- Préparation pour la quantification.

2.4. Cross-Layer Equalization (CLE)

2.4.1. Principe

CLE équilibre les plages dynamiques des poids entre couches consécutives pour réduire l'erreur de quantification. Il combine BatchNorm Folding, Cross-Layer Scaling et High Bias Fold.

2.4.2. Équations

Cross-Layer Scaling : Pour deux couches Conv1 -> ReLU -> Conv2, trouver s tel que :

$$\hat{W}_1 = W_1 \cdot s \text{ et } \hat{W}_2 = \frac{W_2 \cdot 1}{s}$$

High Bias Fold : Ajuster les biais pour compenser les changements d'échelle.

2.4.3. Entrées / Sorties

Élément Type

- Entrée tenseurs de poids float32 (couches consécutives)
- Sortie tenseurs de poids équilibrés float32
- Échelle non applicable
- Décalage z non applicable

2.4.4. Algorithme simplifié

1. Appliquer BatchNorm Folding.
2. Pour chaque paire de couches consécutives (Conv/Linear) :
3. Calculer s pour équilibrer les plages de poids.
4. Mettre à jour $\hat{W}_1$ et $\hat{W}_2$.
5. Ajuster les biais si nécessaire.

2.4.5. Avantages

- Réduction de l'erreur de quantification.
- Améliore la stabilité numérique.
- Aucune donnée requise.

2.4.6. Défauts

- Peut introduire des biais élevés.
- Non applicable à toutes les architectures.

2.4.7. Applications

- Modèles avec déséquilibre de plage (MobileNetV2).
- Pré-quantification.

2.5. AdaScale

2.5.1. Principe

AdaScale est une technique PTQ qui introduit des paramètres entraînables dans les quantificateurs de poids et utilise la distillation bloc par bloc (BKD) pour optimiser les paramètres de quantification. Elle combine des idées de FlexRound et OmniQuant.

Distillation de connaissances

- Définition simple : Petit modèle imitant un grand modèle.

2.5.2. Équations

Pour un bloc, minimiser : $L = \| f_{FP32}(x) - f_{quant}(x) \|^2$

Les poids sont quantifiés avec des paramètres appris (ex. : clipping learnable).

Clipping

- Définition simple : Limitation des valeurs dans un intervalle.
- Définition mathématique :
  - $clip(x, a, b) = min(max(x, a), b)$

2.5.3. Entrées / Sorties

Élément Type

- Entrée tenseur de poids float32, données d'entraînement
- Sortie tenseur quantifié int4/int8 avec paramètres optimisés
- Échelle float (learnable)
- Décalage z entier (learnable)

2.5.4. Algorithme simplifié

1. Créer un QuantSimModel.
2. Pour chaque bloc contigu :
   1. Ajouter des paramètres entraînables (gamma, beta, s2, s3).
   2. Effectuer BKD entre sortie FP32 et quantifiée.
   3. Optimiser avec descente de gradient (≥1500 itérations).

4. Geler les encodages.

2.5.5. Avantages

- Optimisation précise des paramètres de quantification.
- Convient aux modèles complexes (LLM).
- Réduction de l'erreur de quantification.

2.5.6. Défauts

- Expérimental, limité à PyTorch.
- Coûteux en calcul.
- Nécessite des blocs contigus bien définis.

2.5.7. Applications

- Modèles de langage (Llama, Qwen).
- Quantification très bas bit (4 bits).

2.6. OmniQuant

2.6.1. Principe

OmniQuant améliore la précision via deux composantes : Learnable Weight Clipping (LWC) et Learnable Equivalent Transformation (LET). Il utilise BKD pour optimiser les paramètres d'échelle des poids.

2.6.2. Équations

LWC : Introduit un paramètre d'échelle s appris pour clipping : $\hat{w} = clip(w, -s, s)$

LET : Transformations équivalentes pour réduire l'erreur.

Perte : $L = \| y_{block} - \hat{y_{block}} \|^2$

2.6.3. Entrées / Sorties

Élément Type

- Entrée tenseur de poids float32, données
- Sortie tenseur quantifié int4/int8, paramètres s appris
- Échelle float (learnable)
- Décalage z entier

2.6.4. Algorithme simplifié

1. Créer un QuantSimModel.
2. Appliquer apply_omniquant (≥800 itérations).
3. Dumper les échelles apprises (safetensors).
4. Calculer les encodages restants.
5. Exporter le modèle.

2.6.5. Avantages

- Optimisation efficace pour LLM.
- Supporte W4A8, W8A8.
- Réduction d'erreur significative.

2.6.6. Défauts

- Expérimental, limité à PyTorch.
- Nécessite des blocs contigus.
- Temps d’optimisation élevé.

2.6.7. Applications

- Modèles Llama, Qwen, Deepseek.
- Quantification bas bit.

2.7. Batch Norm Re-estimation

2.7.1. Principe

BN Re-estimation recalcule les statistiques (μ, σ) des couches BatchNorm à l'aide d'un sous-ensemble de données d'entraînement, puis les replie. Utile lorsque le repli standard dégrade la précision.

2.7.2. Équations

Re-estimation : $\mu_{new} = mean(y_{batch})\, et\, \sigma^2_{new} = var(y_{batch})$

Puis repli : $\hat{W} = \frac{W \cdot \gamma}{\sqrt{\sigma^2_{new} + \epsilon}}\, et\, \hat{b} = \frac{(b - \mu_{new}) \cdot \gamma}{\sqrt{\sigma^2_{new} + \epsilon}} + \beta$

2.7.3. Entrées / Sorties

Élément Type

- Entrée modèle avec BatchNorm, données d'entraînement
- Sortie modèle avec BN replié et statistiques mises à jour
- Échelle non applicable
- Décalage z non applicable

2.7.4. Algorithme simplifié

1) Effectuer QAT (15-20 epochs).
2) Re-estimer μ, σ avec des données d'entraînement.
3) Replier les BatchNorm.
4) Exporter le modèle.

2.7.5. Avantages

- Améliore la précision après QAT.
- Réduit la sensibilité aux oscillations (couches depth-wise).

2.7.6. Défauts

- Nécessite des données d'entraînement et QAT.
- Non supporté ONNX.

2.7.7. Applications

- Modèles avec BN sensibles (MobileNetV3).
- Post-QAT.

2.8. SpinQuant

2.8.1. Principe

SpinQuant insère des rotations (type R1) à des points spécifiques du modèle pour atténuer les effets des valeurs aberrantes dans les activations. Les rotations peuvent être fusionnées avec les poids adjacents, sans changer l'architecture.

2.8.2. Équations

Rotation $R1$: Pour une matrice de poids W, appliquer une rotation R telle que $\hat{W} = W \cdot R$, où R est une matrice orthogonale.

La sortie devient : $y = x \cdot \hat{W} = x \cdot (W \cdot R) = (x \cdot W) \cdot R$

2.8.3. Entrées / Sorties

Élément Type

- Entrée tenseur de poids float32, modèle avec blocs définis
- Sortie tenseur de poids avec rotation intégrée float32
- Échelle non applicable
- Décalage z non applicable

2.8.4. Algorithme simplifié

1) Enregistrer les paires RMSNorm-Linear et les points d'insertion R1 pour le type de modèle.
2) Appliquer apply_spinquant (fusion RMSNorm et insertion R1).
3) Quantifier le modèle (ex. : QuantSimModel).
4) Calculer les encodages avec données de calibration.
5) Évaluer.

2.8.5. Avantages

- Réduction de l'impact des valeurs aberrantes.
- Modèle final inchangé (rotations fusionnées).
- Compatibilité avec les LLM (Llama, Qwen).

2.9. Défauts

- Expérimental, limité à PyTorch.
- Nécessite une table de mapping pour les points d’insertion.

2.9.1. Applications

- Modèles de langage avec activations aberrantes.
- Quantification W4A16.

2.10. Automatic Quantization (AutoQuant)

2.10.1. Principe

AutoQuant applique automatiquement une séquence de techniques PTQ (BNF, CLE, AdaRound, AMP) jusqu'à atteindre un seuil de précision cible. Il simplifie le processus de quantification en évitant les essais manuels.

2.10.2. Équations

Aucune équation spécifique ; utilise les méthodes sous-jacentes (ex. : AdaRound, CLE).

Objectif : $min||accuracy_{FP32} - accuracy_{quant}|| \leq allowed_{drop}$

2.10.3. Entrées / Sorties

Élément Type

- Entrée modèle FP32, données d'évaluation, callback de précision
- Sortie modèle quantifié optimisé, rapport de précision
- Échelle déterminée automatiquement
- Décalage z déterminé automatiquement

2.10.4. Algorithme simplifié

1. Préparer le modèle et sélectionner le schéma de quantification.
2. Appliquer BNF.
3. Appliquer CLE.
4. Si activé, appliquer AdaRound avec données.
5. Si activé, appliquer AMP (mixed-precision).
6. Évaluer la précision après chaque étape.
7. Arrêter si le seuil est atteint.
8. Retourner le meilleur modèle.

2.10.5. Avantages

- Automatisation complète.

- Optimisation multi-techniques.
- Économise du temps et des efforts.

2.10.6. Défauts

- Non supporté ONNX.
- Peut être coûteux en calcul.

2.10.7. Applications

- Quantification généraliste pour divers modèles.
- Déploiement rapide avec contrainte de précision.

3. MQBench[12,13,14]

3.1. Adaptive Rounding (AdaRound)

3.1.1. Principe

AdaRound détermine la stratégie d'arrondi optimale pour les valeurs quantifiées. L'arrondi au plus proche n'est pas optimal pour des couches entières. La méthode apprend un masque d'arrondi par couche en minimisant la perturbation de la sortie via une fonction de coût avec régularisation.

Régularisation

- Définition simple : Pénalité pour éviter les poids trop grands.
- Définition mathématique :
- L2 régularisation : $L_{reg} = \frac{\lambda}{2} \cdot \sum |w_i|^2$

3.1.2. Équations

Quantification des poids: $\hat{w} = s \times round\left(\frac{w}{s} + v\right)$

Variable d'arrondi: $v = clip\left(0, 1, \sigma(\alpha)\right)$

Fonction de coût: $L = \|f(x, W) - f(x, \hat{W})\|_F^2 + \lambda \times \left(1 - (2v - 1)^2\right)$

Évolution de λ: $\lambda_t = \frac{\lambda_0 \times t}{T}$

3.1.3. Entrées / Sorties

Élément | Type

- Entrée | Poids float32, Données calibration float32
- Sortie | Poids int8/int4, Échelles float32, Masque d'arrondi int64

3.1.4. Algorithme simplifié

1. Initialiser les paramètres α
2. Pour chaque batch de calibration:

3. Calculer les activations flottantes
4. Quantifier les poids avec masque d'arrondi
5. Calculer la perte de reconstruction
6. Mettre à jour α par descente de gradient
7. Geler les poids quantifiés finaux

3.1.5. Avantages

- Optimisation globale par couche
- Meilleure précision que l'arrondi standard
- Convergence binaire garantie

3.1.6. Défauts

- Nécessite données calibration
- Coût computationnel modéré
- Ignore la quantification des activations

3.1.7. Applications

- Réseaux convolutionnels
- Modèles de vision par ordinateur
- Quantification symétrique des poids

3.2. Block Reconstruction Quantization (BRECQ)

3.2.1. Principe

BRECQ étend la reconstruction à des blocs de couches successives. La méthode approxime la matrice hessienne pour reconstruire les sorties bloc par bloc, en considérant les connexions résiduelles.

Matrice Hessienne

- Définition simple : Matrice des dérivées secondes d'une fonction.
- Définition mathématique :
  - $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$

3.2.2. Équations

Perte par bloc: $L_{block} = \| f_{block}(x, W) - f_{block}(x, \hat{W}) \|_2^2$

Optimisation: $\hat{W_{block}} = argmin_{\hat{W_{block}}} L_{block}$

Approximation Hessienne: $H \approx J^T J \text{ où } J = \frac{\partial f_{block}}{\partial W}$

Reconstruction: $\hat{W_{block}} = W_{block} + \Delta W \text{ où } \Delta W = H^{-1} g$

3.2.3. Entrées / Sorties

Élément | Type

- Entrée | Poids float32, Données calibration float32
- Sortie | Poids int8/int4, Échelles float32, Blocs reconstruits

3.2.4. Algorithme simplifié

1. Définir les blocs selon la topologie réseau
2. Pour chaque bloc:
3. Calculer la sortie flottante du bloc
4. Optimiser la quantification des poids du bloc
5. Reconstruire la sortie avec Hessienne approximée

6. Valider sur données de test

3.2.5. Avantages

- Prend en compte les dépendances inter-couches
- Gère les connexions résiduelles
- Meilleure approximation Hessienne

3.2.6. Défauts

- Définition complexe des blocs
- Calcul hessien coûteux
- Sensible aux données calibration

3.2.7. Applications

- Réseaux résiduels (ResNet)
- Architectures complexes
- Quantification très basse (4 bits)

3.3. Quantization with Dropout (QDrop)

3.3.1. Principe

QDrop introduit une quantification stochastique des activations pendant l'optimisation des poids. Cette technique améliore la robustesse en évitant le sur-apprentissage aux données calibration.

Dropout

- Définition simple : Désactivation aléatoire de neurones pendant l'entraînement.
- Définition mathématique :
  - Pour un neurone d'activation a :
  - $\hat{a} = a \cdot m$ où $m \sim Bernoulli(p)$

3.3.2. Équations

Quantification stochastique: $\hat{a} = Q(a) \times B + a \times (1 - B)$ où $B \sim Bernoulli(p)$

Perte espérée: $L = E_{\hat{a}}\left[\|f(x, W, a) - f(x, \hat{W}, \hat{a})\|^2\right]$

Optimisation: $\hat{W} = argmin_{\hat{W}} L$

Probabilité adaptative: $p_t = p_0 \times (1 - \frac{t}{T})$

3.3.3. Entrées / Sorties

Élément | Type

- Entrée | Poids float32, Activations float32, Taux dropout float32
- Sortie | Poids int8/int4, Échelles float32, Modèle robuste

3.3.4. Algorithme simplifié

1. Pour chaque itération:
   1. Sélectionner aléatoirement les couches à quantifier
   2. Appliquer dropout de quantification aux activations
   3. Calculer la perte avec modèle mixte
   4. Mettre à jour les paramètres de quantification

5. Tester la robustesse finale

3.3.5. Avantages

- Réduit le sur-apprentissage
- Robustesse accrue aux perturbations
- Généralisation améliorée

3.3.6. Défauts

- Convergence plus lente
- Paramètre dropout supplémentaire
- Implémentation complexe

3.3.7. Applications

- Environnements bruités
- Quantification très basse précision
- Modèles sensibles aux variations

4. ONNX Runtime[15,16,17,18]

4.1. MinMax Quantization

4.1.1. Principe

La méthode MinMax utilise les valeurs minimales et maximales observées pendant la calibration pour déterminer la plage de quantification. Elle garantit que toutes les valeurs de calibration sont représentables dans la plage entière cible.

4.1.2. Équations

Calcul de l'échelle asymétrique: $s = \frac{x_{max} - x_{min}}{Q_{max} - Q_{min}}$

Calcul du zero-point asymétrique:$z = round\left(Q_{max} - \frac{x_{max}}{s}\right)$

Calcul de l'échelle symétrique: $s = \frac{max\left(|x_{min}|, |x_{max}|\right)}{Q_{max}}$

Zero-point symétrique: $z = 0$

Quantification: $\hat{x} = s \times \left(clip\left(round\left(\frac{x}{s}\right) + z, Q_{min}, Q_{max}\right) - z\right)$

4.1.3. Entrées / Sorties

Élément | Type

- Entrée | Tenseur float32, Q_min int64, Q_max int64, Données calibration float32
- Sortie | Tenseur int8/int4, Échelle float32, Zero-point int32

4.1.4. Algorithme simplifié

1. Collecter les données de calibration
2. Calculer x_min = minimum des valeurs observées
3. Calculer x_max = maximum des valeurs observées
4. Choisir le mode asymétrique ou symétrique
5. Calculer s et z selon les équations

6. Quantifier les poids et activations
7. Vérifier la plage et ajuster si nécessaire

4.1.5. Avantages

- Implémentation simple et rapide
- Aucune valeur hors limites pendant la calibration
- Garantie de couvrir toute la plage observée
- Calculs directs sans optimisation complexe

4.1.6. Défauts

- Très sensible aux valeurs aberrantes
- Mauvaise précision pour distributions asymétriques
- Gaspillage de plage
- Peut dégrader significativement la précision

4.1.7. Applications

- Données uniformément distribuées
- Prototypage rapide et validation de concept
- Environnements avec contraintes de calcul limitées
- Cas où la simplicité prime sur la précision

4.2. Entropy (KL Divergence) Quantization

4.2.1. Principe

La méthode Entropy utilise la divergence de Kullback-Leibler pour minimiser la perte d'information entre la distribution flottante originale et la distribution quantifiée. Elle détermine un seuil de clipping optimal qui préserve la forme de la distribution.

4.2.2. Équations

Divergence KL: $D_{KL}(P||Q)=\sum_i P(i)\log\left(\frac{P(i)}{Q(i)}\right)$

- Distribution originale P: histogramme des valeurs flottantes avant quantification
- Distribution quantifiée Q: histogramme des valeurs après quantification-déquantification

Construction de Q: $Q(i)=\sum_j P(j)\times I(dequant(quant(x_j))\,dans\,bin\,i)$

Fonction indicatrice $I(condition)=1\,si\,condition\,vraie\,,0\,sinon$

Seuil optimal: $t^{¿}=argmin_t D_{KL}(P\vee ¿Q_t)$

Échelle: $s=\frac{t}{Q_{max}}$

Quantification: $\hat{x}=s\times clip(round(\frac{x}{s}),-Q_{max},Q_{max})$

4.2.3. Entrées / Sorties

Élément | Type

Entrée | Tenseur float32, Données calibration float32, Nombre bins int64

Sortie | Tenseur int8/int4, Échelle float32, Seuil optimal float32

4.2.4. Algorithme simplifié

1. Collecter les activations sur données calibration
2. Construire l'histogramme P des valeurs
3. Pour chaque candidat seuil t:
4. Calculer l'échelle $s=t/Q_{max}$

5. Quantifier et déquantifier les valeurs
6. Construire l'histogramme Q des valeurs reconstruites
7. Calculer D_{KL}(P || Q)
8. Sélectionner le seuil t minimisant D_{KL}
9. Calculer les paramètres de quantification finaux

4.2.5. Avantages

- Préserve la forme de la distribution originale
- Meilleure précision pour distributions non uniformes
- Gestion optimisée du compromis saturation/erreur de quantification

4.2.6. Défauts

- Calcul intensif pendant la calibration
- Sensible au nombre de bins de l'histogramme
- Plus complexe à implémenter que MinMax

4.2.7. Applications

- Distributions d'activations asymétriques
- Modèles sensibles à la forme de distribution
- Cas où la précision est critique et le temps de calibration secondaire

5. Microsoft NNI[33]

Microsoft NNI (Neural Network Intelligence) est un projet open-source de Microsoft Research conçu pour simplifier et automatiser l'expérimentation en intelligence artificielle.

Il aide chercheurs, data scientists et ingénieurs à trouver plus facilement les meilleurs modèles et réglages pour leurs projets de machine learning, en automatisant notamment le réglage d'hyperparamètres, la recherche d'architectures neuronales (NAS) et la compression de modèles.

NNI s'utilise aussi bien sur une simple machine locale que sur des environnements plus complexes comme des serveurs distants, Kubernetes ou Azure.

Flexible et compatible avec de nombreux frameworks connus (PyTorch, TensorFlow, Scikit-learn, XGBoost, etc.), il offre une interface web pratique pour suivre ses expériences.

Hébergé sur GitHub et soutenu par une large communauté, NNI évolue rapidement et propose une documentation riche, des exemples concrets et des ressources vidéo pour accompagner les utilisateurs dans leurs projets, qu'ils soient débutants ou experts.

5.1. Naive Quantizer

5.1.1. Principe

Le Naive Quantizer est une méthode de quantification basique qui convertit les poids en entiers 8 bits sans configuration avancée. Il utilise une quantification symétrique avec l'arrondi au plus proche.

5.1.2. Équations

Échelle: $s = \frac{max(|w|)}{127}$

Quantification: $\hat{w} = s \times round\left(\frac{w}{s}\right)$

Arrondi: $round(x) = \lfloor x + 0.5 \rfloor$

Plage: $\hat{w}\ dans\ [-127\,s, 127\,s]$

5.1.3. Entrées / Sorties

Élément | Type

- Entrée | Poids float32, Modèle non quantifié
- Sortie | Poids int8, Échelles float32, Modèle quantifié

5.1.4. Algorithme simplifié

1. Parcourir toutes les couches du modèle
2. Pour chaque tenseur de poids:
3. Calculer s = max(|w|) / 127
4. Quantifier: w_quant = round(w / s)
5. Stocker w_quant et s
6. Remplacer les poids par leurs versions quantifiées

5.1.5. Avantages

- Implémentation extrêmement simple
- Aucun hyperparamètre à régler
- Rapide pour les tests initiaux

5.1.6. Défauts

- Précision souvent médiocre
- Sensible aux outliers
- Aucune optimisation de la plage

5.1.7. Applications

Prototypage rapide

Tests de validation conceptuelle

Benchmark de base pour comparaison

5.2. DoReFa Quantizer

5.2.1. Principe

DoReFa quantifie les poids, activations et gradients pendant l'entraînement avec différentes largeurs de bits. La méthode utilise des fonctions de quantification non linéaires adaptées à l'apprentissage.

5.2.2. Équations

Quantification poids: $\hat{w} = 2 \times quantize_k\left(\frac{\tanh(w)}{2\,max(|\tanh(w)|)} + \frac{1}{2}\right) - 1$

Quantification activations: $\hat{a} = quantize_k(a)$

Quantification gradients: $\hat{g} = quantize_k(g)$

$$quantize_k(x) = \frac{1}{2^k - 1} round\left((2^k - 1)x\right)$$

5.2.3. Entrées / Sorties

Élément | Type

Entrée | Poids float32, Activations float32, Gradients float32

Sortie | Poids int1-int8, Activations int1-int8, Gradients int1-int8

5.2.4. Algorithme simplifié

1. Configurer les bits pour poids/activations/gradients
2. Pendant l'entraînement:
3. Avant forward: quantifier les poids avec fonction tanh
4. Pendant forward: quantifier les activations
5. Pendant backward: quantifier les gradients
6. Mettre à jour les poids avec gradients quantifiés

5.2.5. Avantages

- Quantification très basse précision (1-2 bits)
- Entraînement from scratch possible
- Optimisation conjointe poids/activations/gradients

5.2.6. Défauts

- Convergence parfois difficile
- Fonctions de quantification complexes
- Hyperparamètres sensibles

5.2.7. Applications

- Réseaux très compressés
- Accélération matérielle extrême
- Apprentissage embarqué

5.3. BNN Quantizer

5.3.1. Principe

Le BNN Quantizer binarise les poids et activations en valeurs +1 ou -1, remplaçant les opérations arithmétiques par des opérations bit-à-bit. La méthode utilise des approximations straight-through pour l'estimation des gradients.

5.3.2. Équations

Binarisation: $\hat{w} = sign(w) = \begin{cases} +1 \\ \text{si } w \geq 0 \\ -1 \text{¿si } w < 0 \text{¿} \end{cases}$

Activation binaire: $\hat{a} = sign(a)$

Estimateur gradient: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{w}} \times 1_{|w| \leq 1}$

5.3.3. Entrées / Sorties

Élément | Type

Entrée | Poids float32, Activations float32

Sortie | Poids int1, Activations int1, Gradients float32

5.3.4. Algorithme simplifié

(1) Sélectionner les couches à binariser

(2) Pendant l'entraînement:

(3) Forward: w_bin = sign(w), a_bin = sign(a)

(4) Calculer la perte avec valeurs binaires

(5) Backward: utiliser l'estimateur straight-through

(6) Mettre à jour les poids avec gradients réels

5.3.5. Avantages

- Réduction drastique mémoire (32x)
- Opérations bit-à-bit très efficaces
- Accélération matérielle importante

5.3.6. Défauts

- Perte de précision significative
- Convergence difficile sur tâches complexes
- Gestion délicate des gradients

5.3.7. Applications

- Classification d'images simples
- Déploiement sur hardware contraint
- Recherche sur modèles extrêmement efficaces

6. Neural Magic SparseML [34,35]

6.1. Introduction

SparseML est un framework open-source développé par Neural Magic spécialisé dans l'optimisation de modèles de deep learning via la sparsification (élagage et quantification). Il s'intègre à des bibliothèques comme PyTorch et permet d'accélérer l'inférence sur des CPUs standard.

6.2. Origine, objectifs et positionnement

Créé pour démocratiser l'optimisation des modèles, SparseML vise à obtenir des performances GPU sur CPU via des techniques avancées de compression. Bien que son cœur de métier soit le Quantization-Aware Training (QAT) (quantification pendant l'entraînement), il prend également en compte des méthodes de post-training quantization (PTQ) via des approches one-shot.

6.2.1. Principales fonctionnalités

Recettes de sparsification prédéfinies (élagage, quantification) pour YOLOv5 et autres architectures.

- Intégration transparente avec PyTorch et export vers ONNX.
- 3 méthodes clés :
  - One-shot (PTQ dynamique sans réentraînement)
  - Sparse Transfer Learning (recyclage de modèles sparse existants)
  - Training Aware (QAT avec élagage et quantification conjointe)
- Accélération matérielle via le moteur DeepSparse pour l'inférence CPU.

6.3. Conclusion

SparseML excelle dans le QAT et l'élagage, mais son utilisation pour la PTQ pure repose davantage sur des frameworks comme ONNX ou PyTorch déjà documentés. Bien que marginalement pertinent pour une étude centrée sur la PTQ, il reste un outil notable pour les pipelines hybrides alliant compression avancée et optimisation CPU.

7. Nebullvm[36]

7.1. Introduction

Nebullvm est un outil open-source d'accélération de l'inférence IA qui automatise l'optimisation des modèles en testant systématiquement différentes techniques de compression et compilateurs.

7.2. Origine, objectifs et positionnement

Développé par Nebuly, il vise à fournir une accélération maximale des modèles avec un minimum de code. Son positionnement est celui d'un méta-outil qui orchestre plusieurs solutions d'optimisation existantes plutôt que de développer ses propres méthodes de quantification.

7.2.1. Principales fonctionnalités

- Sélection automatique des meilleures techniques d'optimisation selon les contraintes (précision, temps)
- Pipeline modulaire en 4 étapes : Conversion → Compression → Optimisation → Inférence
- Support multi-frameworks : PyTorch, TensorFlow, ONNX, JAX
- Intégration de compilateurs : TensorRT, OpenVINO, ONNX Runtime, TVM, etc.
- Mécanisme de compression qui s'appuie principalement sur :
  - Intel Neural Compressor (déjà cité)
  - SparseML (déjà cité)

7.3. Conclusion

Bien que Nebullvm soit un outil complet d'accélération, nous ne nous y attardons pas dans ce rapport car sa valeur ajoutée réside dans l'automatisation et l'orchestration de méthodes existantes plutôt que dans le développement de nouvelles approches PTQ. Il utilise en backend les mêmes frameworks déjà analysés (SparseML, Neural Compressor) pour la quantification, sans apporter d'innovation méthodologique significative dans le domaine spécifique de la PTQ.

8. AMD Quark[37,38]

8.1. Introduction

AMD Quark est une boîte à outils complète de quantification pour le deep learning, développée par AMD pour optimiser les modèles en vue de leur déploiement sur divers matériels.

8.2. Origine, objectifs et positionnement

Créé par AMD pour fournir une solution de quantification cross-platform, Quark vise à simplifier l'optimisation des modèles PyTorch et ONNX tout en maintenant un équilibre entre performance et précision. Son positionnement est celui d'une solution enterprise avec un support matériel étendu.

8.2.1. Principales fonctionnalités

- Support dual framework : quantification native pour PyTorch et ONNX
- Méthodes de quantification :
  - PTQ (Post-Training Quantization)
  - QAT (Quantization-Aware Training)
- Types de données étendus : int4, uint4, fp8, formats MX (Microscaling)
- Algorithmes spécialisés LLM : SmoothQuant, AWQ, GPTQ, QuaRot
- Intégration matérielle AMD via APL (AMD Pytorch-light)
- Export multi-formats : ONNX, JSON-safetensors, GGUF

8.3. Conclusion

AMD Quark représente une solution enterprise mature pour la quantification, mais nous ne l'analysons pas en détail car elle s'appuie fondamentalement sur les mêmes frameworks de base (PyTorch, ONNX) déjà documentés dans ce rapport. Bien qu'offrant des implémentations optimisées pour le hardware AMD, ses méthodologies de quantification PTQ restent conceptuellement similaires à celles des autres outils étudiés.

9. PyTorch[20,21,22]

9.1. Origine et contexte

PyTorch est un framework open-source d'IA, conçu pour programmer des réseaux de neurones, notamment en Python. Il est issu de la bibliothèque Torch, puis adapté pour Python et GPU, et développé par la division recherche de Meta (anciennement Facebook) : « Facebook AI Research ».

9.2. Pourquoi utiliser PyTorch ?

Quelques raisons majeures :

- Le framework est très bien intégré au langage Python, ce qui facilite la prise en main pour les personnes maîtrisant Python.
- Il y a une large communauté et un écosystème riche : modules spécialisés, modèles pré-entraînés, bibliothèques complémentaires (comme torchvision, torchtext)
- Pour les projets de recherche ou d'optimisation de modèles (comme les CNN), il offre une grande flexibilité : débogage plus naturel, adaptation rapide.

9.3. Domaines d'application

PyTorch couvre un large spectre :

- Vision par ordinateur (classification d'images, détection d'objets, segmentation) , il va à programmer de simples algorithmes de régression linéaire jusqu'aux réseaux neuronaux convolutifs complexes, pour des tâches telles que la vision par ordinateur et le traitement automatique du langage naturel (NLP)
- Traitement du langage naturel (NLP), etc.
- Donc, pour nos architectures CNN, PyTorch est tout à fait pertinent.

9.4. Architecture, composants et réglages dans PyTorch

9.4.1. Principaux composants

Voici les composants structurels qu'il faut connaître pour manipuler un modèle CNN dans PyTorch :

- Tenseurs : La donnée de base dans PyTorch est le tenseur (multidimensionnel/manipulation des données) qui peut être sur CPU ou GPU.
- Module torch.nn : La bibliothèque de couches pré-définies (convolutions, pooling, fonctions d'activation, etc.).
- Autograd (torch.autograd) : Mécanisme qui permet de calculer les gradients et de faire la rétropropagation.
- DataLoader et Dataset (torch.utils.data) : Pour la gestion des jeux de données, batching, shuffle, etc
- Compilateur / exportation (torch.jit, TorchScript) : Pour l'export ou le déploiement.

9.4.2. Mise en œuvre d'un CNN

Dans un projet CNN typique avec PyTorch :

1. Préparation des données (prétraitement, normalisation, transformations, DataLoader).
2. Définition de l'architecture du modèle (héritage de nn.Module, définition __init__ et forward) : couches nn.Conv2d, nn.ReLU, nn.MaxPool2d, nn.Linear, etc.
3. Choix de la fonction de perte (ex. nn.CrossEntropyLoss) et de l'optimiseur (ex. torch.optim.SGD, Adam).
4. Boucle d'entraînement (forward, loss, backward, step de l'optimiseur) et évaluation sur un jeu de validation.
5. Réglages : taux d'apprentissage (learning rate), dropout, momentum, augmentation de données, choix de l'architecture (nombre de couches, taille des filtres) etc.
6. Déploiement ou exportation du modèle (ex. via TorchScript) si nécessaire.

9.4.3. Réglages et optimisations spécifiques

Pour tirer pleinement parti de PyTorch et optimiser les CNN, plusieurs leviers sont à considérer :

- Choix de l'optimiseur : SGD, momentum, Adam, etc. Le package torch.optim fournit ces modules.

- Augmentation de données (data augmentation) : pour éviter le sur-apprentissage.
- Batch size, nombre d'epochs : ces hyperparamètres ont un effet significatif sur l'entraînement des CNN.
- Transfert d'apprentissage : utiliser un modèle pré-entraîné (torchvision.models) puis le régler sur notre tâche, ce qui accélère l'entraînement et peut améliorer la performance.

9.4.4. Compatibilité d'architectures CNN

PyTorch convient à une large gamme d'architectures CNN : des réseaux relativement simples (ex. LeNet, AlexNet) à des architectures plus complexes (ex. ResNet, VGG, MobileNet). L'écosystème torchvision propose plusieurs modèles pré-entraînés. De plus, via PyTorch, on peut implémenter des architectures personnalisées, expérimenter des modifications. Cela offre une grande flexibilité pour adapter une architecture à notre problème

9.5. Synthèse

En résumé :

- PyTorch est un framework fiable, flexible et largement adopté pour le deep learning, et particulièrement adapté aux CNN.
- Il offre de nombreux réglages pour l'entraînement d'un CNN (choix de l'architecture, hyperparamètres, GPU, optimiseur, etc).
- Concernant la quantisation, PyTorch propose des workflows robustes (PTQ, QAT) permettant de réduire la taille et d'accélérer les modèles pour l'inférence, ce qui est particulièrement pertinent si notre objectif est l'optimisation (latence, mémoire) du modèle CNN.

9.6. Conclusion

Le framework PyTorch constitue un excellent choix pour l'étude et la mise en œuvre de CNN, tant pour la phase d'entraînement que pour l'optimisation (y compris quantisation). Il combine souplesse, puissance, et support de quantisation avancé.

10. INC[23,24,25,26]

10.1. Présentation

Intel Neural Compressor (INC) est une bibliothèque open-source visant à automatiser et standardiser les techniques de compression (quantisation, pruning, distillation) pour les principaux frameworks (TensorFlow, PyTorch, ONNX Runtime).

10.2. Qu'est-ce que Intel Neural Compressor

- Origine / qui l'a fait : projet initié et maintenu par Intel (répertoire public intel/neural-compressor sur GitHub). Le dépôt centralise le code et la documentation.

- Pourquoi : répondre au besoin d'outils pour réduire la taille et la latence des modèles tout en contrôlant la perte .INC vise aussi à standardiser les pipelines (PTQ, QAT, pruning, distillation) afin de faciliter l'adoption pour les ingénieurs.

10.3. Fonctionnalités principales

- Techniques supportées : quantisation (INT8, INT4, FP8), quantisation PTQ et QAT, pruning, distillation, neural architecture search (NAS), et outils spécifiques pour LLMs.

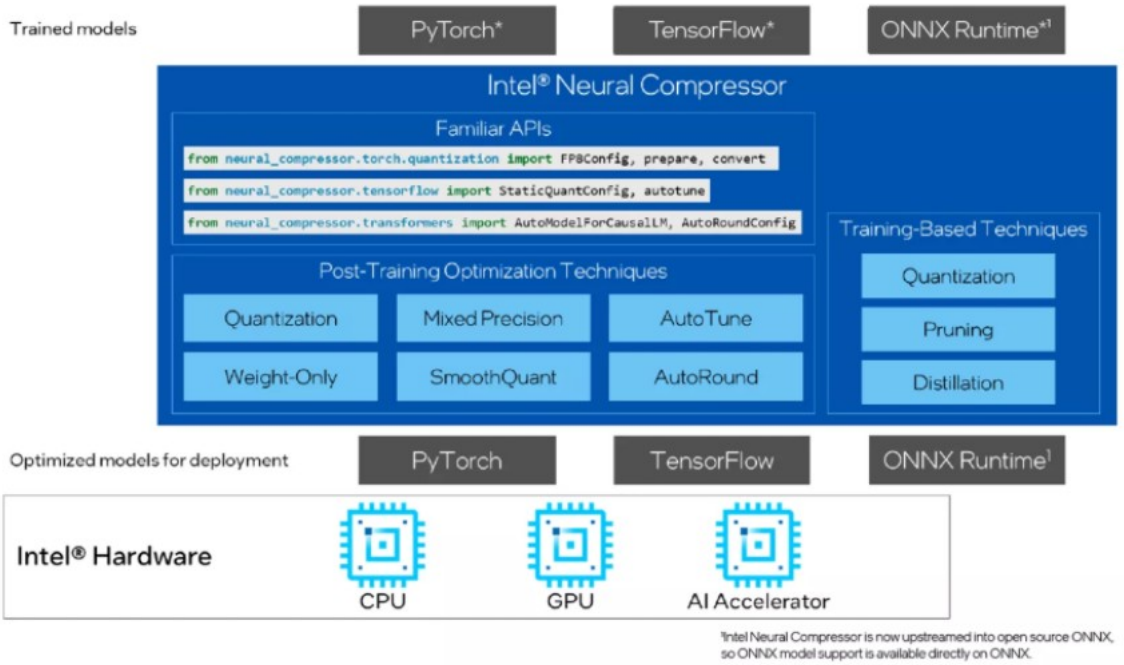
- Multi-framework : intégration native avec TensorFlow, PyTorch et ONNX Runtime.

- Automatisation et tuning : INC propose un moteur de tuning : on définit des contraintes (ex. perte d'accuracy max autorisée, objectifs de latence), INC exécute des stratégies de compression et renvoie la meilleure configuration respectant les contraintes. Ceci réduit le travail manuel de calibration.

- Support matériel : tests et optimisations forts pour CPU Intel, Intel GPU et accélérateurs ; mais INC peut aussi cibler d'autres architectures (ARM, AMD, NVIDIA) via ONNX Runtime avec un niveau de test limité.

10.4. Techniques d'optimisations Post-Training :

Dans le cadre du post-training de modèles, INC offre plusieurs techniques d'optimisation majeures : mixed precision, quantisation, auto-tune, weight-only quantisation (WOQ), SmoothQuant, AutoRound. Ces techniques peuvent être combinées ou utilisées individuellement en fonction des contraintes.



- Mixed precision consiste à utiliser différentes précisions numériques (par exemple FP32 + FP16 ou INT8 + FP16) dans le même modèle, de façon à réduire les exigences de calcul/mémoire tout en conservant une précision acceptable. INC fournit un support pour le mixed precision inference/training via ses APIs.

- Quantisation classique (PTQ) est prise en charge : réduction des poids et/ou activations à des représentations plus petites (INT8, INT4, ou formats FP8 selon matériel), réduction de la taille du modèle et de la latence d'inférence. INC permet l'automatisation de cette quantisation.

- AutoTune fait référence à l'algorithme d'INC qui explore automatiquement les configurations de quantisation/pruning/mixed-precision pour respecter un critère d'accuracy-tolérance et un objectif matériel (ex : latence) sans que l'utilisateur doive manuellement tester toutes les options.

- Weight-Only Quantisation (WOQ) est une technique spécialisée où seul le jeu de poids du modèle est quantifié (souvent en INT4 ou INT8) tandis que les activations restent en plus haute précision ou non quantifiées, ce qui peut être particulièrement avantageux pour les très grands modèles (LLM) ou dans des scénarios embarqués.

- SmoothQuant

- AutoRound est une méthode d'arrondi optimisé (« rounding ») pour les poids quantifiés, afin de minimiser l'erreur induite par la quantisation .

|

En combinant ces techniques, INC permet de passer d'un modèle de référence en float32 à une version compressée/quantifiée optimisée pour l'inférence.

10.5. Intégration avec PyTorch (points pratiques)

- Tutoriels: PyTorch fournit un tutoriel dédié « Ease-of-use quantization for PyTorch with Intel Neural Compressor » expliquant l'intégration et les exemples pour PyTorch. INC expose une API Python et des fichiers de configuration pour régler la quantisation depuis PyTorch.

- Extensions recommandées : pour des cibles CPU Intel, il est conseillé d'installer les extensions Intel. Ces extensions activent des kernels optimisés et garantissent de meilleurs gains de performance.

10.6. Conclusion

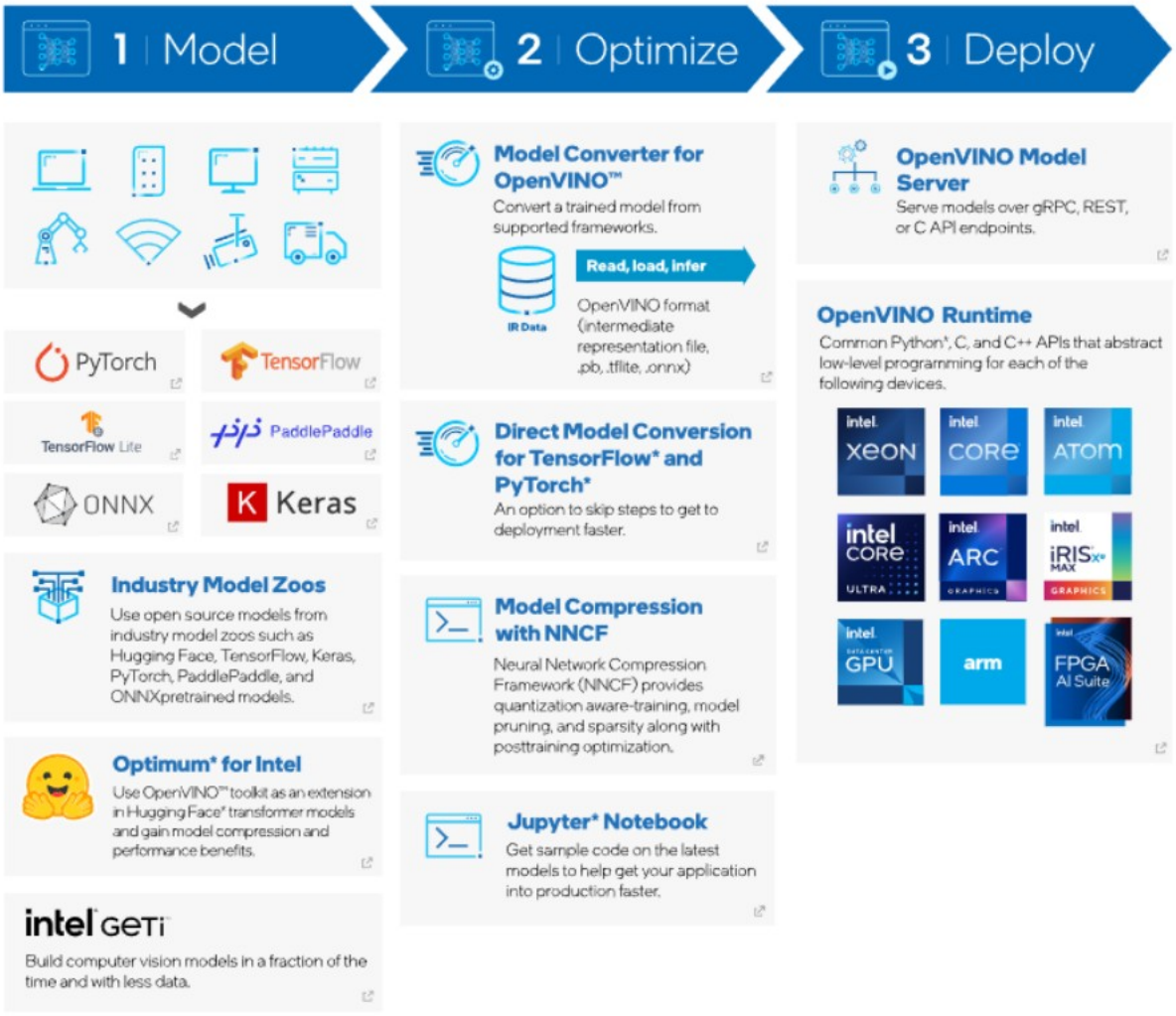
Intel Neural Compressor est une boîte à outils robuste et moderne pour la compression et l'optimisation de modèles, particulièrement adaptée aux enjeux d'un déploiement industriel sur hardware Intel mais aussi utilisable via ONNX Runtime pour d'autres cibles. Pour nos CNN, INC permet d'automatiser PTQ/QAT, d'explorer des compromis précision/latence et d'intégrer ces étapes dans nos pipelines de quantisations.

11. OpenVINO[27,28,29]

11.1. Présentation de OpenVINO Toolkit

11.1.1. Origine et contexte

OpenVINO signifie “Open Visual Inference and Neural network Optimization”. C’est un toolkit open-source développé par Intel, conçu pour optimiser et déployer des modèles de deep learning (y compris CNN) sur une large gamme de matériels (CPU Intel, GPU Intel)



11.1.2. Pourquoi utiliser OpenVINO ?

Quelques raisons majeures :

- Le framework est pensé pour optimiser l'inférence: conversion de modèles, optimisation, déploiement.
- Il couvre un large éventail de matériels Intel.
- Il propose des outils d'optimisation (quantisation, pruning, conversion) qui sont précieuses pour réduire la latence et le coût mémoire des modèles CNN.
- Pour les architectures CNN classiques (et d'autres domaines comme la NLP) il existe un "Model Zoo" prêt à l'emploi et optimisé.

11.1.3. Domaines d'application

OpenVINO s'inscrit particulièrement bien dans les taches suivantes :

- Vision par ordinateur (classification d'images, détection d'objets, segmentation).
- Traitement du langage naturel (NLP), grands modèles, réseaux de vision multimodaux.

11.2. Architecture, composants et réglages dans OpenVINO

11.2.1. Principaux composants

Voici les composants structurels qu'il faut connaître pour manipuler un modèle CNN avec OpenVINO :

- Model Optimizer : outil de conversion d'un modèle entraîné dans un framework (TensorFlow, PyTorch, ONNX, etc.) vers le format intermédiaire OpenVINO (IR). Cette conversion permet les optimisations de graphe, fusion d'opérations, changement de type de données.
- Intermediate Representation (IR) : format interne d'OpenVINO contenant structure et poids optimisés du modèle pour l'inférence.
- Inference Engine / Runtime OpenVINO : runtime qui exécute le modèle optimisé sur le matériel cible (CPU, GPU).
- NNCF (Neural Network Compression Framework) : module d'OpenVINO (ou intégré) permettant des optimisations avancées comme quantisation aware, pruning, weight compression.
- Post-Training Optimization Tool : outil de quantisation post-training qui fonctionne sur le modèle IR pour appliquer quantisation, calibration et optimiser l'inférence.

11.2.2. Mise en œuvre d'un CNN

Dans un projet CNN typique avec OpenVINO, le flux est :

1. Entraîner un modèle CNN (ex. ResNet, VGG, YOLO) dans un framework standard (PyTorch, TensorFlow).
2. Convertir le modèle dans le format OpenVINO avec le Model Optimizer en spécifiant l'entrée, la taille, éventuellement le type de données.
3. Appliquer des optimisations (quantisation, pruning) via NNCF ou POT pour obtenir un modèle optimisé pour l'inférence.
4. Charger le modèle optimisé dans l'Inference Engine OpenVINO sur le matériel cible, choisir la cible (CPU, GPU) et exécuter l'inférence.
5. Ajuster les réglages pour la performance : type de données (FP32, FP16, INT8), batch size, précision, etc.
6. Vérifier la précision, la latence, et la consommation/mémoire.

11.2.3. Réglages et optimisations spécifiques

Pour tirer pleinement parti d'OpenVINO et optimiser les CNN, plusieurs leviers sont à considérer :

- Choix du type de données : FP32 est la précision par défaut mais FP16, INT8 sont disponibles selon cible matérielle. Cela peut apporter des gains de performance.
- La cible : il faut indiquer le matériel de destination.
- Optimisation de graphe : lors de la conversion, on peut fusionner certaines couches, éliminer des opérations inutiles, redimensionner via Model Optimizer.
- Batch size & threading : pour l'inférence sur CPU, ajuster le nombre de threads, et ajuster le nombre de data dans la batch.

11.2.4. Compatibilité d'architectures CNN

OpenVINO convient à une large gamme d'architectures CNN : des réseaux relativement simples (ex. LeNet, AlexNet) à des architectures plus complexes (ResNet, VGG, YOLO, SSD). Le Model Zoo d'Intel propose des architectures de détection (face-detection) compatibles.

De plus, puisque OpenVINO supporte plusieurs formats (ONNX, TensorFlow, PyTorch via export) et permet l'optimisation de l'IR, on peut adapter quasiment toute architecture CNN standard pour l'inférence optimisée.

11.3. Synthèse

En résumé :

- OpenVINO Toolkit est un framework fiable, flexible et largement adopté pour le deep learning, et particulièrement adapté aux CNN et aux usages d'inférence/optimisation surtout pour les cibles intel.
- Il offre de nombreux réglages pour l'entraînement d'un CNN (via conversion, optimisation d'inférence, quantisation, pruning).
- Concernant la quantisation et l'optimisation, OpenVINO propose des workflows robustes (Post-Training Quantization, QAT, Weight Compression) permettant de réduire la taille et d'accélérer les modèles pour l'inférence.

11.4. Conclusion

Le framework OpenVINO Toolkit constitue un excellent choix pour l'étude et la mise en œuvre de CNN, tant pour la phase d'entraînement (via conversion) que pour l'optimisation (quantisation, pruning, déploiement). Il combine souplesse, puissance, et support de quantisation avancé.

12. Brevitas[30,31,32]

12.1. Introduction

Brevitas est une bibliothèque PyTorch dédiée à la quantisation des réseaux de neurones (supportant à la fois Post-Training Quantization et Quantization-Aware Training). Elle vise le monde de la recherche et du prototypage quantifié : fournir des primitives et couches quantisées (QuantConv, QuantLinear, QuantMultiheadAttention, etc.), des quantizers déclaratifs et une API permettant d'expérimenter finement les espaces de paramètres de quantisation (bit-width, granularité des scales, signed/unsigned, etc.). Brevitas est maintenue sous l'égide de Xilinx.

12.2. Origine, objectifs et positionnement

- Qui / origine : le dépôt officiel est Xilinx/brevitas sur GitHub ; le projet est présenté par Xilinx comme une librairie open-source pour la quantisation dans l'écosystème PyTorch. Le README précise aussi que Brevitas est un projet de recherche (et non un produit « officiel »).

- Pourquoi : fournir un plateau expérimental complet pour concevoir, tester et affiner des stratégies de quantisation (bit-width, échelle par canal, types d'arrondi, quantize-aware training, etc.) avant de passer à l'étape de déploiement. Brevitas se concentre sur la spécification fine des quantizers et la réutilisabilité des blocs quantifiés.

- Positionnement : outil de recherche/prototypage pour quantisation (PTQ & QAT) directement intégré à PyTorch - il n'est pas un runtime d'inférence matériel-spécifique, mais il facilite la génération de modèles quantisés prêts à être exportés vers chaînes de déploiement (ONNX, etc.) ou vers d'autres toolchains d'implémentation matérielle.

12.3. Principales fonctionnalités (ce que Brevitas apporte)

- Couches quantisées (brevitas.nn) : implémentations prêtes à l'emploi de couches utilisées - QuantConv2d, QuantLinear, QuantConvTranspose2d, QuantMultiheadAttention, QuantRNN, QuantLSTM, etc. Ces couches permettent de contrôler la quantisation des entrées, poids, biais et sorties de façon indépendante.

- Support PTQ & QAT : workflows et exemples pour la quantisation après entraînement (PTQ) et pour la quantification pendant l'entraînement (QAT). Des notebooks et exemples ImageNet sont fournis comme référence.

- Quantizers déclaratifs et configurables : Brevitas propose des quantizers modulaires (choix du bit-width, échelle per-channel/per-tensor, signed vs unsigned, narrow range, etc.) permettant d'explorer systématiquement l'effet des choix de quantification.

- Contrôle fin du comportement numérique : options d'arrondi, de clipping, et de gestion des zéros (zero-point) pour reproduire et tester différents formats numériques (INT8, INT4, formats spécialisés).

12.4. Architectures et cas d'usage ciblés

- Architectures couvertes : toute architecture implémentée via les primitives PyTorch (CNN classiques : ResNet, VGG, MobileNet, réseaux de détection comme SSD/YOLO) et architectures séquentielles/attention (RNN/LSTM, Transformer/Multi-Head Attention) - Brevitas fournit des couches pour ces blocs.

- Scénarios d'utilisation :

  - prototypage et évaluation d'architectures CNN quantifiées ;

  - études comparatives PTQ vs QAT, recherche de bit-widths optimaux ;

  - préparation de modèles quantisés pour export (ONNX) afin d'être déployés ensuite sur une chaîne d'implémentation matérielle (FPGA/ASIC).

12.5. Réglages importants à maîtriser (pratiques)

Brevitas est conçu pour donner accès à de nombreux hyper-réglages de la quantisation. Les plus importants à connaître et à documenter lors d'un protocole d'évaluation sont :

1. Bit-width (quantization bit-width) : choisir le nombre de bits pour poids et activations (INT8, INT4, etc.). Impact direct sur taille/latence et précision.

2. Granularité de la scale : per-tensor vs per-channel (per-channel souvent préférable pour les poids convolutionnels pour réduire la perte de précision).

3. Signed vs Unsigned / Narrow range : définir si la représentation inclut un signe et si la plage est réduite (narrow range)

4. Méthode d'initialisation des scales / calibration : pour PTQ il faut une calibration (jeu d'images représentatif) ; pour QAT l'initialisation et le comportement des fake-quantize modules pendant l'entraînement sont déterminants.

5. Stratégie d'arrondi / clipping : choix d'arrondi (round-to-nearest, stochastic rounding, AutoRound-like techniques) et stratégie de clipping des activations pour réduire l'erreur de quantisation. Brevitas permet d'expérimenter ces comportements.

6. Quantification sélective (which tensors to quantize) : quantifier uniquement les poids (weight-only), ou aussi activations/outputs/inputs ; ce choix est clé pour les très grands modèles (LLM) ou scénarios embarqués.

7. Compatibilité training/inference : paramétrer les modules pour QAT (fake quantize pendant l'entraînement) ou pour PTQ (appliquer quantization sur modèle entraîné).

12.6. Limites et points de vigilance

- Projet de recherche : Brevitas est pensé pour la recherche et le prototypage ; il n'est pas un runtime d'inférence optimisé pour production. Pour le déploiement sur FPGA / ASIC ou runtimes spécifiques, il faut compléter avec des toolchains de déploiement (export / conversion / mapping vers Vitis/FINN/ONNX Runtime ou équivalent).

- Compatibilité PyTorch : exigences sur les versions de PyTorch - vérifier la compatibilité avant intégration.

- Complexité expérimentale : large nombre d'options et de combinaisons → nécessité d'un protocole expérimental strict (calibration, jeux de validation, métriques reproductibles) pour comparer.

12.7. Conclusion

Brevitas est un outil pertinent pour prototyper et étudier finement des stratégies de quantisation (PTQ & QAT) dans l'écosystème PyTorch. Sa granularité de contrôle et son intégration simple au workflow PyTorch en font un choix naturel pour des études sur CNN (et pour préparer des versions quantifiées avant export vers une chaîne de déploiement matérielle).

13. Projet « Awesome Model Quantization »[19]

13.1. Introduction

Le "Awesome Model Quantization" est un projet open source sur GitHub qui fournit une collection exhaustive de ressources et d'outils concernant la quantification des modèles d'IA. La quantification est un processus crucial dans l'optimisation des modèles, car elle permet de réduire la taille des modèles, d'améliorer la vitesse d'exécution et de diminuer la consommation énergétique tout en maintenant la performance. Ce processus est particulièrement important pour les déploiements sur des systèmes embarqués avec des ressources limitées.

Le dépôt GitHub regroupe des articles de recherche, des outils, des bibliothèques et des frameworks pour aider les ingénieurs à comprendre et à appliquer les techniques de quantification aux réseaux de neurones. Ce projet vise à fournir un ensemble de ressources de haute qualité, adaptées à un large éventail de techniques de quantification. Pour recenser et analyser les différentes méthodes PTQ, le dépôt GitHub Awesome Model Quantization (https://github.com/Efficient-ML/Awesome-Model-Quantization?tab=readme-ov-file#bibench) constitue une source centralisée et structurée.

13.2. Objectif du dépôt GitHub

L'objectif principal de ce dépôt est de centraliser les différentes méthodes et outils relatifs à la quantification des modèles. Il fournit une liste de frameworks et d'outils qui permettent de réduire la précision des poids et des activations des modèles, en passant d'une représentation flottante (32 bits) à des représentations entières (8 bits, par exemple).

Le dépôt vise aussi à être un guide pour les personnes qui cherchent à optimiser les modèles en proposant des ressources éducatives, des articles scientifiques et des liens vers des outils d'optimisation. L'objectif est également de fournir une comparaison des différentes approches, de manière à ce que les utilisateurs puissent choisir la méthode la plus adaptée à leurs besoins.

Le dépôt a été créé par Efficient ML, et son objectif est de centraliser toutes les ressources importantes liées à la quantification des modèles :

- Articles scientifiques sur les méthodes PTQ et QAT (quantization-aware training).
- Benchmarks pour comparer les méthodes de quantification.
- Frameworks et outils open-source pour appliquer la quantification.
- Ressources sur la quantification adaptée au matériel (« hardware-aware quantization »).

13.3. Structure du github

- Le dépôt est organisé de manière à rendre l'exploration des différentes ressources et outils aussi intuitive que possible. Voici une description générale de sa structure :

13.3.1. README :

- Introduction au projet et à son but.
- Liens vers les principales ressources disponibles.
- Explications des concepts fondamentaux de la quantification des modèles.

13.3.2. Table des matières :

- Liste de sections concernant les outils, les méthodes, et les frameworks de quantification.

13.3.3. Méthodes de quantification :

- Détail des différentes techniques de quantification, y compris la Post-Training Quantization (PTQ), et d'autres approches.

13.3.4. Frameworks et Outils :

- Un répertoire de frameworks et d'outils populaires pour la quantification des modèles.

13.3.5. Références académiques :

- Une compilation des articles de recherche qui ont contribué aux avancées dans le domaine de la quantification des modèles.
- Ces articles donnent un aperçu global des techniques de quantification, leurs principes et les avancées récentes.

13.3.6. Projets associés :

- Liste de projets open-source ou de bibliothèques populaires qui peuvent être utilisées en complément pour effectuer de la quantification de modèles.

13.4. Revue des méthodes PTQ

13.4.1. Exemples de méthodes PTQ:

BRECQ (Block Reconstruction Post-Training Quantization)

- Cible : Modèles CNN
- Avantages : La méthode BRECQ permet de quantifier sans avoir à retrainer le modèle et préserve la précision des modèles CNN, en particulier ceux de type ResNet.
- Inconvénients : Plus efficace pour les CNNs, mais ne s'applique pas encore aux modèles de type Transformer ou LLM (Large Language Models).

GPTQ (Generalized Post-Training Quantization)

- Cible : Modèles Transformers et LLM (comme GPT, BERT).
- Avantages : Optimisation pour les quantifications en int 4 avec une faible perte de précision, spécifiquement adaptée aux Transformers.
- Inconvénients : La méthode est spécifique aux modèles Transformers et n'est pas aussi efficace pour les autres types de modèles.

SmoothQuant

- Cible : LLMs (ex : GPT-3, BERT).
- Avantages : Cette méthode corrige les activations et permet de quantifier en int4/int8 avec de bons résultats pour les grands modèles.
- Inconvénients : Complexe à mettre en œuvre, nécessite des ajustements fins pour fonctionner efficacement.

13.4.2. Benchmarks :

MQBench : Un outil essentiel pour évaluer les performances des différentes techniques de quantification. Ce benchmark permet de mesurer les impacts de la quantification sur la précision et la taille du modèle de manière reproductible, sur des

modèles comme les CNNs et Transformers. Il permet de choisir la méthode la mieux adaptée à un modèle ou à un matériel spécifique.

13.4.3. Hardware-aware Quantization :

Des ressources spécifiques sont également disponibles pour adapter la quantification en fonction du matériel, ce qui permet d'optimiser la performance et la précision des modèles sur différents types de processeurs (ex : ARM, GPU).

13.4.4. Conclusion

Le dépôt GitHub "Awesome Model Quantization" est une ressource pour les personnes qui souhaitent optimiser les modèles pour des applications sur des dispositifs avec des ressources limitées. En centralisant les méthodes, les outils et les frameworks de quantification, il permet aux utilisateurs de découvrir et d'appliquer les techniques les plus appropriées à leurs besoins spécifiques.

14. Conclusion et Choix Technologiques

Pour déterminer les frameworks et méthodes à retenir pour nos expérimentations, nous avons appliqué une grille de sélection basée sur deux critères majeurs : la pérennité du projet (activité du dépôt GitHub et maintenance) et la présence de méthodes de quantification *Post-Training* (PTQ) complètes et intégrées.

Nous avons ainsi écarté les solutions agissant comme de simples "surcouches" (wrappers) ou celles dont l'usage est trop spécifique (recherche pure ou matériel propriétaire exclusif) au profit de solutions industrielles robustes.

Voici la synthèse de notre sélection :

Framework	AIMET	MQBench	ONNX Runtime	Microsoft NNI	Neural Magic SparseML	Nebullvm	AMD Quark	PyTorch	INC	OpenVINO	Brevitas
Activité Github											
Methode unique											

14.1. Synthèse de la sélection

À la lumière de cette analyse, trois frameworks se distinguent pour une application industrielle fiable : **AIMET**, **ONNX Runtime** et **Intel Neural Compressor (INC)**. Ils offrent le meilleur compromis entre maturité des outils et performance des méthodes de quantification.

14.2. Prochaines étapes

Conformément à cette sélection, nous débuterons notre phase expérimentale par **ONNX Runtime**. Ce choix est motivé par sa simplicité de mise en œuvre, sa large compatibilité et sa capacité à fournir rapidement une "baseline" (référence) de performance avant d'explorer des méthodes plus complexes via AIMET ou INC

ANNEXES

TABLE DES FIGURES

TABLE DES TABLEAUX

RÉFERENCES

[1] https://github.com/quic/aimet
[2] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/index.html
[3] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/adaround.html
[4] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/seq_mse.html
[5] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/bnf.html
[6] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/cle.html
[7] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/adascale.html
[8] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/bn.html
[9] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/omniquant.html
[10] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/autoquant.html
[11] https://quic.github.io/aimet-pages/releases/latest/ptq_techniques/spinquant.html
[12] https://github.com/ModelTC/MQBench
[13] https://mqbench.readthedocs.io/en/latest/user_guide/algorithm/index.html
[14] https://mqbench.readthedocs.io/en/latest/user_guide/algorithm/advanced_ptq.html
[15] https://github.com/onnx/onnx
[16] https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html
[17] https://quark.docs.amd.com/latest/onnx/config/calibration_methods.html
[18] https://apxml.com/courses/practical-llm-quantization/chapter-2-post-training-quantization-ptq/common-ptq-algorithms
[19] https://github.com/Efficient-ML/Awesome-Model-Quantization
[20] https://www.nexa.fr/post/pytorch-tout-savoir-sur-ce-logiciel-open-source
[21] https://www.ibm.com/fr-fr/think/topics/pytorch
[22] https://github.com/pytorch/pytorch
[23] https://github.com/intel/neural-compressor
[24] https://www.intel.com/content/www/us/en/developer/tools/oneapi/neural-compressor.html#tab-blade-1-1
[25] https://www.intel.com/content/www/us/en/developer/articles/technical/an-easy-introduction-to-intel-neural-compressor.html
[26] https://docs.pytorch.org/tutorials/recipes/intel_neural_compressor_for_pytorch.html
[27] https://github.com/openvinotoolkit/openvino
[28] https://www.intel.com/content/www/us/en/developer/tools/openvino-toolkit/overview.html
[29] https://docs.openvino.ai/2025/index.html
[30] https://github.com/Xilinx/brevitas
[31] https://xilinx.github.io/brevitas/v0.12.1/
[32] https://www.amd.com/en/developer/resources/technical-articles/open-source-quantization-with-advanced-data-formats.html
[33] https://nni.readthedocs.io/en/v2.1/Compression/Quantizer.html
[34] https://github.com/neuralmagic/sparseml
[35] https://dicksonneoh.com/portfolio/supercharging_yolov5_180_fps_cpu
[36] https://github.com/towhee-io/nebullvm
[37] https://www.amd.com/en/developer/resources/technical-articles/amd-quark-quantizer-for-efficient-ai-model-deployment.html
[38] https://quark.docs.amd.com/latest/
[39]

ENSI CAEN

ÉCOLE PUBLIQUE D'INGÉNIEURS
CENTRE DE RECHERCHE

Ecole Publique d'Ingénieurs en 3 ans

6 boulevard Maréchal Juin, CS 45053
14050 CAEN cedex 04

ministère éducation nationale enseignement supérieur recherche
Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

Normandie Université

Cti Commission des titres d'ingénieur

cdefi Conférence des Directeurs des Écoles Françaises d'Ingénieurs

European Accreditation of Engineering Programmes EUR-ACE®

CONFÉRENCE DES GRANDES ÉCOLES